CISS350: Data Structures and Advanced Algorithms
Test t01

Name: eagraham1@cougars.ccis.edu Score:

Instructions

- This is a closed-book, no-discussion, no-calculator, no-browsing-on-the-web no-compiler test.
- Cheating is a serious academic offense. If caught you will receive an immediate score of -100%.
- If a question asks for a program output and the program or code fragment contains an error, write `ERROR` as output. When writing output, whitespace is significant.
- If a question asks for the computation of a value and the program or code fragment contains an error, write `ERROR` as value.
- When you're asked to write a C++ statement, don't forget that it must end with a semicolon.
- Unless otherwise stated, bubblesort, selectionsort, insertionsort, mergesort, and quicksort refers to the algorithms mention in class. Unless otherwise stated, sorting is always in ascending order.

To enter your answers, look for `answerbox` or `answerlong` in `main.tex`. If you have problems compiling, let me know. When you are done:

- Submit using alex.
- Email me `yliow@ccis.edu` with `submit.tar.gz` attached and with subject line `"ciss350 t01"`.

Honor Statement

I, [Ethan Alaric Graham], attest to the fact that the submitted work is my own and is not the result of plagiarism. Furthermore, I have not aided another student in the act of plagiarism.

Question	Points
1	
2	
3	
4	
5	
6	
7	
8	
9	
10	
11	
12	
13	
14	
15	
16	
17	
18	
19	
20	

Question	Points
21	
22	
23	
24	
25	
26	
27	
28	
29	
30	
31	
32	
33	
34	
35	
36	
37	
38	
39	
40	

TOTAL	

Some LaTeX

For $T(n) = (t_0 + t_1)n^2 + t_2n + t_3$ write `$T(n) = (t_0 + t_1)n^{2} + t_2n + t_3$`.

For $T(n) = O(n^{100})$ write `$T(n) = O(n^{100})$`.

Let me know if you have questions about LaTeX.

Note that $T(n) = n^2$ is not the same as $T(n) = O(n^2)$.

Q1.

The following computes the sum of squares from 1^2 to n^2:

```
s = 0
for i = 1, ..., n:
    term = i * i
    s = s + term
```

Here's the program with goto statements and timing for each statement:

```
         i = 1              time t0  1
         s = 0              time t1  1
LOOP:    if i > n:          time t2  n + 1
             goto ENDLOOP   time t3  1
         term = i * i       time t4  n
         s = s + term       time t5  n
         i = i + 1          time t6  n
         goto LOOP          time t7  n
ENDLOOP:
```

(a) Compute the time taken as a function of n. The expression should contain constants t_0, ..., t_7.
Answer:

$T(n) = (t_2 + t_4 + t_5 + t_6 + t_7)n + (t_0 + t_1 + t_2 + t_3)$

(b) Simplify the runtime function by giving names A, B, ... to the constants of the function from (a).
Answer:

$T(n) = An + B$

(c) Fudge away the constants and write down the simplest $g(n)$ such that the time in (b) is a big-O of your $g(n)$. Your $g(n)$ should be either n^0 (i.e., 1) or n or n^2 or n^3 or ...
Answer:

$T(n) = O(n)$

Q2.

If the asymptotic runtime cannot be computed based on only what is taught in CISS350, write ERROR in the answer. Note that the algorithms are written in pseudocode (as usual).

(a) What is the big–O of the runtime of the following algorithm:

```
int f(int x[], int n)
{
    s = 0
    for i = 0, 1, 2, ..., n - 1:
        for j = 0, 1, 2, ..., i:
            for k = j, j + 1, ..., n - 1:
                s = s + x[i][j] + k
    return s
}
```

Answer:

$T(n) = O(n^3)$

(b) What is the big–O of the runtime of the following algorithm:

```
int f(int x[], int n)
{
    s = 0
    for i = 0, 1, 2, ..., n - 1:
        for j = n - 3, n - 2, n - 1, n:
            for k = i, i + 1, ..., n - 1:
                s = s + x[i][k] * j
    return s
}
```

Answer:

$T(n) = O(n^2)$

(c) What is the big–O of the best runtime of the following algorithm:

```
int f(int x[], int n)
{
    s = 0
    for i = 0, 1, 2, ..., n - 1:
        for j = n/2, n/2 + 1, ..., n:
            if x[0][0] > 0:
                for k = i, i + 1, ..., n - 1:
                    s = x[j][k] + x[i][k]
    return s
}
```

Answer:

$T(n) = O(n^2)$

(d) What is the big–O runtime of the following algorithm:

```
int f(int x[], int start, int end)
{
    n = end - start
    if n <= 1:
        return 42 + x[start]
    else:
        if x[0] == 42:
            return 24
        else:
            mid = start + n / 2
            if x[0] < 42:
                return f(x, start, mid)
            else:
                return f(x, mid, end)
}
```

Answer:

$T(n) = O(lgn)$

Q3. For questions on runtime, just state the big-O of the runtime performance.

(a) What is the runtime of insertion sort?
Answer:

$T(n) = O(n^2)$

(b) What is the worst runtime of mergesort?
Answer:

$T(n) = O(nlgn)$

(c) What is the best runtime of mergesort?
Answer:

$T(n) = O(nlgn)$

(d) What is the average runtime of mergesort?
Answer:

$T(n) = O(nlgn)$

(e) What is the worst runtime of quicksort?
Answer:

$T(n) = O(n^2)$

(f) What is the best runtime of quicksort?
Answer:

$T(n) = O(nlgn)$

(g) What is the average runtime of quicksort?
Answer:

$T(n) = O(nlgn)$

(h) What algorithm would you use to sort an array of 1000000000 doubles with values > 10 and < 1000000? (Assume you want the fastest of course.)
Answer:

I would use a counting sort algorithm

Q4. Perform insertion sort on the following array

```
{5, 1, 2, 4, 3, 6}
```

showing the result of each pass (i.e., after each insert).
Answer:

```
{1, 5, 2, 4, 3, 6} - pass 1
{1, 2, 5, 4, 3, 6} - pass 2
{1, 2, 4, 5, 3, 6} - pass 3
{1, 2, 3, 4, 5, 6} - pass 4
```

(You need not use all of the above arrays. Remove those not used. Add more arrays if you need to.)

Q5. You are given the following singly linked node class:

```
class SLNode
{
public:
    SLNode(int key, SLNode * next=NULL)
    : key_(key), next_(next)
    {}

    int key_;
    SLNode * next_;
};
```

Note that all members are public (to make life simple). The following is an incomplete singly listed list class for integer values, `SLList`. Complete the following methods `insert_head()` and `delete_head()`. Assume there is an `UnderflowError` class that can be used for exception throwing under a suitable condition.

```
class SLList
{
public:
    SLList()
      : phead_(NULL)
    {}
    void insert_head(int key);
    void delete_head();
    // ... other methods
private:
    SLNode * phead_;
};
```

(a) Answer:

```
void SLList::insert_head(int key)
{
  phead_ = new SLNode(key, phead_);
}
```

(b) Answer:

```
void SLList::delete_head()
{
  SLNode * q = phead_;
  phead_ = phead_->next_;
  delete q;
}
```

Q6. State the big-O of the runtime of the following operations in a singly linked list (n is the size of the list).

(a) Insert head:
Answer:

$T(n) = O(1)$

(b) Delete head:
Answer:

$T(n) = O(1)$

(c) Insert tail:
Answer:

$T(n) = O(n)$

(d) Delete tail:
Answer:

$T(n) = O(n)$

(e) Delete first node with a given key value:
Answer:

$T(n) = O(n)$

(f) Compute the maximum of all the key values in the list:
Answer:

$T(n) = O(n)$

(g) Swap the keys of the first two nodes (if there are at least two nodes):
Answer:

$T(n) = O(1)$

(h) Insert a given key after the node of a given pointer:
Answer:

$T(n) = O(1)$

(i) Insert a given key before the node of a given pointer:
Answer:

$T(n) = O(1)$

Q7. Assume the following functions for performing quicksort are provided:

```
// Selects the index of a pivot in x[start .. end - 1]
int pivot_index(int x[], int start, int end);

// Performs a partition algorithm on x[start .. end - 1] with the pivot at
// x[i]. The return value is the index of the pivot at the end of the
// partition algorithm.
int partition(int x[], int start, int end, int i);
```

Complete the quicksort below.

Answer:

```
void quicksort(int x[], int start, int end)
{
    if(end - start <= 1)
      return;

    int pivotindex = pivot_index(x, start, end);
    int pivotfinal = partition(x, start, end, pivotindex);

    quicksort(x, start, pivotfinal);
    quicksort(x, pivotfinal + 1, end);
}
```

Q8. You are given the following array:

```
{3, 4, 0, 1, 7, 6, 8, 9, 2}
```

(a) Write down the array after performing the median-of-three pivot selection and write down the pivot value.
Answer:

```
{2, 4, 0, 1, 3, 6, 8, 9, 7}
```

Pivot value is

```
3
```

(b) Write down the array after performing one partitioning. (As mentioned in class, there are many possible partitioning algorithms. You must use the method I that mentioned in class, the method I called pLRT.)
Answer:

```
{2, 0, 1, 3, 4, 6, 8, 9, 7}
```

Q9. An array x of size n contains `doubles` -2.0, -1.9, -1.8, ..., 0.1, 0, 0.1, ..., 2.0. Note that n is huge, approximately 1000000000. Explain how you would sort this array. Note that you want the fastest possible algorithm. Explain briefly how you would sort this array and provide an implementation below (the name of the function is `sort`). In your explanation, include the runtime and how much extra memory you need. The extra memory does not include the array and the size of the array. But everything must be stated, even index variables used in loops. Complete the following sorting function.

Answer:

(a) Explanation:

To sort this array I want to implement a counting sort algorithm. In a for-loop: to handle the decimals I will multiply each element of the array by 10 and add 20 to handle negatives, I will change these values back later with another for-loop. This will give me a count range from 0 to 40. Create a loop that puts the number of times each value occurs to the corresponding index in count. Then, make a value int j, int i, and double k all set to 0. In a double for-loop, iterate i until it reaches 40 (our range), on the inner-loop, k will stop when it is greater than count[i]. Within this loop, x[j] will equal i. After sorting through the entire array, convert back to a range of -2.0 to 2.0. The runtime for this should be O(n + k), with memory needed being O(k), k being the range of numbers.

(b) Implementation:

```
void sort(double x[], int n)
{
  for(int i = 0; i < n; i++)
  {
    x[i] *= 10;
    x[i] += 20;
  }

  double count[40] = { 0.0 };
  for(int i = 0; i < n; i++)
  {
    count[x[i]]++;
  }
  int j = 0;
  for(int i = 0; i < 40; i++)
  {
    for(double k = 0.0; k < count[i]; k++)
    {
      x[j] = double(i);
      j++;
    }
```

```
  }

   for(int i = 0; i < n; i++)
  {
    x[i] -= 20;
    x[i] /= 10;
  }
}
```

Q10. Complete the following mergesort and merge functions. Assume that the given temporary array is sufficiently large for the merge function.

Answer:

```
int t[1000000000]; // Temporary array that can be used by merge()

// Performs the merge operation on x[start0 .. end0 - 1] and
// x[start1 .. end1 - 1]. Here start1 = end0. You can assume the left and right
// subarray can fit into the above temporary array t.
void merge(int x[], int start0, int end0, int start1, int end1)
{
  int left = start0, right = start1, k = 0;

  while(left < end0 && right < end1)
  {
    if(x[left] < x[right])
      t[k++] = x[left++];
    else
      t[k++] = x[right++];
  }

  while(left < end0)
     t[k++] = x[left++];
  while(right < end1)
     t[k++] = x[right++];

  for(int i = 0; i < k; i++)
  {
    x[start + i] = t[i];
  }
}

// Performs mergesort on x[start..end - 1]
void mergesort(int x[], int start, int end)
{
  if(end - start <= 1)
    return;

  int mid = start + (end - start) / 2;

  mergesort(x, start, mid);
  mergesort(x, mid, end);

  merge(x, start, mid, mid, end);
}
```

Q11. The following array has gone through one complete partitioning step of the quicksort algorithm:

```
{3, 2, 0, 4, 1, 5, 8, 6, 9}
```

The partitioning algorithm and the pivot selection algorithm used are not known and might not be the ones you have seen before. Write down all values which might be pivots?

Answer:

```
4,5,9
```

(Edit the above. I'm not claiming there are exactly 3 possible pivots.)

Q12. Mergesort is performed on the following array:

```
{4, 2, 0, 3, 1, 5, 8, 6}
```

Of course we know that mergesort is recursive and calls itself. When we *return* from the *first* mergesort function call, the array is completed sorted. What are the values of the array when we *return* from the *third* mergesort function call.

Answer:

```
{2, 4, 0, 3, 1, 5, 8, 6}
```

Q13. Suppose it is known that the array `x` has even indexed values which are ascending and odd indexed values which are ascending as well. For instance in the array `{5,0,8,1,9,2,10,7}`, the values at even indexed positions are `5,8,9,10` which are in ascending order; the values at odd indexed positions are `0,1,2,7` which are also in ascending order. The array can have a huge size and the values need not be a small range.

How would you sort such an array? (Of course you want the fastest algorithm that uses the least amount of extra space, with speed being more important.)

Explain briefly (and clearly) your method. Include your runtime and space complexity. Next, implement the sorting function; the name of the function is `sort`. A temporary array `t` (using `std::vector`) is declared in the sorting function. If you do not need it, you should remove it.

Answer:

Explanation:

I would make a temporary vector array called it t. I will created two separate for loops, with the first loop putting all even indexed values into t, and the second doing the same but for odd indexed values. Move this semi-sorted array back into x and empty out t. Now, perform a merge of the even and odd portions. I will need an even and odd index for this, as well as a index for t, call it k. Put the lesser element of the two even odd indexes into t and repeat until either index has to stop. Use two while-loops to place any remaining elements into t from either the even or odd index. Last, copy everything from t back into x.
The runtime is $O(?)$ and the space complexity is $O(?)$.

Implementation:

```
// n is the size of the array
void sort(std::vector< int > & x, int n)
{
    std::vector< int > t(n);

    //fill t with even indexed values first
    int j = 0;
    for(int i = 0; i < n; i = i + 2)
    {
      t[j] = x[i];
      j++
    }
    //fill t with odd indexed values
    for(int i = 1; i < n; i = i + 2)
```

```
    {
      t[j] = x[i];
      j++;
    }

    for(int i = 0; i < n; i++)
    {
      x[i] = t[i];
    }

    t.empty();

    int even = 0, odd = n/2, k = 0;

    while(even < n/2 && odd < n)
    {
      if(x[even] < x[odd])
        t[k++] = x[even++];
      else
        t[k++] = x[odd++];
    }

    while(even < n/2)
       t[k++] = x[even++];
    while(odd < n/2)
       t[k++] = x[odd++];

    for(int i = 0; i < k; i++)
    {
      x[i] = t[i];
    }
}
```

Instructions

In the file `thispreamble.tex` look for

```
\renewcommand\AUTHOR{}
```

and enter your email address:

```
\renewcommand\AUTHOR{jdoe5@cougars.ccis.edu}
```

(This is not really necessary since alex will change that for you when you execute `make`.) In your bash shell, execute "`make`" to recompile `main.pdf`. Execute "`make v`" to view `main.pdf`.

Enter your answers in `main.tex`. In the bash shell, execute "`make`" to recompile `main.pdf`. Execute "`make v`" to view `main.pdf`.

For each question, you'll see boxes for you to fill. For small boxes, if you see

```
1 + 1 = \answerbox{}.
```

you do this:

```
1 + 1 = \answerbox{2}.
```

`answerbox` will also appear in "true/false" and "multiple-choice" questions.

For longer answers that need typewriter font, if you see

```
Write a C++ statement that declares an integer variable name x.
\begin{answercode}
\end{answercode}
```

you do this:

```
Write a C++ statement that declares an integer variable name x.
\begin{answercode}
int x;
\end{answercode}
```

`answercode` will appear in questions asking for code, algorithm, and program output. In this case, indentation and spacing is significant. For program output, I do look at spaces and newlines.

For long answers (not in typewriter font) if you see

```
What is the color of the sky?
\begin{answerlong}
\end{answerlong}
```

you can write

```
What is the color of the sky?
\begin{answerlong}
The color of the sky is blue.
\end{answerlong}
```

A question that begins with "T or F or M" requires you to identify whether it is true or false, or meaningless. "Meaningless" means something's wrong with the question and it is not well-defined. Something like "$1 + 2 = 4$" is either true or false (of course it's false). Something like "$1+_2 = 4$?" does not make sense.

When writing results of computations, make sure it's simplified. For instance write 2 instead of $1 + 1$.

Higher level classes.

For students beyond 245: You can put LaTeX commands in `answerlong`.

More examples of meaningless statements: Questions such as "Is $42 = 1+_2$ true or false?" or "Is $42 = \{2\}^{\{3\}}$ true or false?" does not make sense. "Is $P(42) = \{42\}$ true or false?" is meaningless because $P(X)$ is only defined if X is a set. For "Is $1 + 2 + 3$ true or false?", "$1 + 2 + 3$" is well–defined but as a "numerical expression", not as a "proposition", i.e., it cannot be true or false. Therefore "Is $1 + 2 + 3$ true or false?" is also not a well-defined question.

More examples of simplification: When you write down sets, if the answer is $\{1\}$, do not write $\{1, 1\}$. And when the values can be ordered, write the elements of the set in ascending order. When writing polynomials, begin with the highest degree term.

When writing a counterexample, always write the simplest.